

Insights Engine: Prompt-Metrics Explainer

This document provides a comprehensive breakdown of the metrics returned by the `/domains/{domain_id}/prompt-metrics` endpoint. It maps the mathematical definitions from the [LaTeX Specification \(docs/insights_engine_full_latex.md\)](#) to the actual implementation in the codebase.

1. Metric Definitions & Mathematics

1.1 Visibility (Prompt-Level)

- **Term:** `visibility` (0–100%)
- **Meaning:** How visible a brand is in the LLM's response for a specific prompt, relative to the most visible brand for that same prompt.
- **Math (Eq. 17, 25, 29):**
 1. **Mention Score (MS):** $MS = Presence * (1/Rank) * \log(1 + Frequency)$.
 2. **Raw Visibility (Vis_raw):** $Vis_raw = w_p * MS$ (where w_p is prompt weight, defaults to 1.0).
 3. **Cross-Model Aggregation:** $Vis_raw_agg = \Sigma (\alpha_m * Vis_raw_m)$ across all models (GPT-4, Claude, etc.).
 4. **Normalization:** $Visibility \% = 100 * (Vis_raw_agg) / (Max_Vis_raw_agg_in_prompt + \epsilon)$.
- **Implementation:**
 - Logic: `app/engine/metrics/visibility.py` -> `prompt_visibility()`
 - Normalization: `app/services/prompt_metrics_query.py` (lines 241–252)

1.2 Visibility Rank

- **Term:** `visibility_rank`
- **Meaning:** The dense rank of the focal brand among all detected brands for this prompt (1 = most visible).
- **Math (Eq. 32):** $Rank = 1 + \text{count of brands with strictly higher } Vis_raw_agg$.
- **Implementation:**
 - Logic: `app/engine/metrics/visibility.py` -> `dense_rank_descending()`
 - Linked: `app/services/prompt_metrics_query.py` -> `vis_ranks` (line 263)

1.3 Share of Voice (SOV)

- **Term:** `sov` (0–100%)
- **Meaning:** The brand's "slice of the pie" regarding total visibility mass for that prompt.

- **Math (Eq. 34):** $SoV = 100 * Vis_raw_agg / (\sum All_Brands_Vis_raw_agg + \epsilon).$
- **Implementation:**
 - Logic: `app/engine/metrics/sov.py -> share_of_voice()`
 - Linked: `app/services/prompt_metrics_query.py` (line 260)

1.4 Position Metrics

- **Terms:** `avg_rank`, `avg_pos_score`, `position_rank`
- **Meaning:**
 - `avg_rank`: The weighted average numerical position (1st, 2nd, etc.).
 - `avg_pos_score`: Token-normalized prominence (1.0 = start of response, 0.0 = end).
 - `position_rank`: Dense rank based on `avg_rank` (1 = best/lowest rank).
- **Math (Eq. 36-39):**
 - **Conditioning:** Average only includes cases where the brand was actually present (`Presence=1`).
 - **Ranking:** Ascending dense rank (lower rank value is better).
- **Implementation:**
 - Logic: `app/engine/metrics/position.py`
 - Linked: `app/services/prompt_metrics_query.py` (lines 213, 226, 266)

1.5 Sentiment

- **Term:** `sentiment` (-1.0 to 1.0)
- **Meaning:** The weighted average tone associated with the brand mentions.
- **Math (Eq. 40-41):**
 - $Sent = \sum (\alpha_m * w_p * Sent_m * Presence) / (\sum \alpha_m * w_p * Presence + \epsilon).$
 - Conditioned on presence to avoid "Not Mentioned" being treated as "Neutral (0)".
- **Implementation:**
 - Logic: `app/engine/metrics/sentiment_metrics.py -> cross_model_sentiment()`
 - Linked: `app/services/prompt_metrics_query.py` (line 227)

2. Implementation Assumptions & Choices

Due to underspecification in the original LaTeX docs, the following engineering assumptions were made:

1. **Epsilon (ϵ):** Set to `1e-9`. Used to prevent division by zero in empty generations.
2. **Prompt-Level Normalization:** While the LaTeX defines Visibility normalization (Eq. 29) for the *Brand-Level* dashboard, the API also applies it per-prompt so that the "Leader" for every single prompt always shows as 100% in the UI.
3. **Model Weights (α_m):** Defaults are loaded from `settings.default_alpha_m`. If not configured, models are weighted equally.

4. **Logarithmic Frequency:** Frequency weight is strictly $\log(1 + \text{count})$. This prevents a brand with 100 repetitive mentions from "breaking" the visibility scale against a brand with 1 high-quality mention.

3. Edge Cases & Inaccuracies

3.1 The "Empty Response" Paradox

If **no brands** are mentioned in a response:

- **Visibility:** All brands get **0.0%**.
- **Rank:** All brands get **Rank 1** (Tied).
- **Why:** The math $1 + \text{count}(\text{brands} > \text{self})$ results in $1 + 0 = 1$. This can be misleading as it looks like the focal brand is "winning" even though it's not there.

3.2 Missing Data / Absent Focal Brand

- If the **Focal Brand** is absent:
 - `avg_rank` and `position_rank` are returned as `None` (or 0.0 in some schemas) because they are conditioned on presence.
 - `visibility` is 0.0.
- If a **Competitor** is absent:
 - They are filtered out of the `brands_mentioned` list entirely to save payload size.

3.3 Tokenization Variance

The `avg_pos_score` depends on the token count of the response. Since different models (GPT vs Claude) use different tokenizers, `avg_pos_score` is a **heuristic proxy** rather than an absolute physical measurement when aggregated across models.

4. Code Architecture Map

Layer	Responsibility	Key Files
API / Query	Fetching snapshots, late-stage normalization, formatting for JSON.	<code>app/services/prompt_metrics_query.py</code>
Orchestrator	Running the math pipeline for a batch of generations.	<code>app/engine/metrics/pipeline.py</code>
Math Primitives	Atomic functions for SOV, Visibility, Position, etc.	<code>app/engine/metrics/*.py</code>
Data Model	Database schema for <code>metric_snapshots</code> .	<code>app/models/metric.py</code>